

MetaGPT-Mini

An LLM-agnostic reimplementation of
MetaGPT (ICLR 2024 Oral, #1 LLM-Agent)

A learning guide for the M.Tech class

Jai Kumar Meena - M.Tech by Research, CSE
Delhi Technological University

<https://github.com/mannuking/metagpt-mini>

Five papers. One implemented. Zero closed-source models.

1. Why I Picked MetaGPT Out of the Five

I had to choose **one paper** to teach in class tomorrow out of five famous agentic-engineering papers, all from top-tier venues (NeurIPS, ICLR, COLM). Here is the comparison and the reasoning.

The five papers at a glance

MetaGPT - Multi-agent orchestration with SOPs. PM, Architect, Engineer, QA collaborating via a shared message pool. (*ICLR 2024 Oral - top 1.2% of all submissions, ranked #1 in the LLM-Agent category.*)

SWE-agent - Agent-Computer Interfaces for autonomous software engineering. State-of-the-art on SWE-bench at the time of publication. (*NeurIPS 2024.*)

OpenHands (OpenDevin) - Production-grade agent runtime. Terminals, browsers, editors, code interpreters in one harness. (*COLM 2024 Oral.*)

Reflexion - Self-improving agent memory via verbal reinforcement. (*NeurIPS 2023.*)

Voyager - Lifelong skill acquisition: an agent that grows its own tool library. (*NeurIPS 2023.*)

Why MetaGPT won

- **Pedagogical completeness.** MetaGPT is the only paper that models an entire organization (PM + Architect + Engineer + QA). Five other layers (orchestration, message passing, role specialization, SOP encoding, output verification) all appear in one paper.
- **Reproducibility in 1 hour.** The data flow is message-passing, not heavy machinery like Docker (OpenHands), reinforcement-learning training loops (Reflexion), or skill-library code-execution sandboxes (Voyager). A student can read and implement MetaGPT in 1 hour; the others take 1 day.
- **Real citations.** 50k+ GitHub stars on the original, hundreds of follow-up papers, ICLR Oral - no marketing fluff needed.
- **Maps to the 2026 frontier.** The harness-engineering papers I researched (arxiv 2603-2609) all explicitly cite MetaGPT as the multi-agent baseline. Teaching MetaGPT is teaching the foundation the new 2026 work builds on.

What MetaGPT does NOT do (where the others win)

- SWE-agent beats it on raw SWE-bench scores (better agent-computer interface).
- OpenHands is the production-grade runtime that would actually deploy MetaGPT.
- Reflexion adds self-reflection - MetaGPT does not learn from past runs.
- Voyager grows skills continuously - MetaGPT is stateless across runs.

All of those are *additive*: you can bolt Reflexion self-reflection or Voyager skill library onto MetaGPT. MetaGPT is the substrate.

2. The Four Other Papers - Context

Even though MetaGPT is what we implement, you should be able to place it next to its peers on a slide and answer: why not the other? Here is the at-a-glance summary of each.

Paper	Venue	Year	Core idea	Best demo time
MetaGPT	ICLR 2024 Oral	2024	Multi-agent SOP orchestration (PM/Arch/Eng/QA)	1 minute
SWE-agent	NeurIPS 2024	2024	Agent-Computer Interface design pattern	5 minutes (needs Docker)
OpenHands	COLM 2024 Oral	2024	Production-grade agent harness runtime	10+ min (full Docker stack)
Reflexion	NeurIPS 2023	2023	Verbal-RL: agents self-reflect from failure	1 minute
Voyager	NeurIPS 2023	2023	Lifelong skill acquisition + curriculum	5+ min (needs Minecraft env)

Each paper in 60 seconds

SWE-agent (NeurIPS 2024)

Yang et al. noticed that the LLM is fine, but the *interface* matters more. A custom Agent-Computer Interface (ACI), with short commands and scoped outputs, lets GPT-4 solve 12.29% of SWE-bench issues end-to-end. The key insight: do not make the LLM type shell commands; give it purpose-built verbs.

OpenHands (COLM 2024 Oral)

Formerly OpenDevin. A platform that runs an agent with: a bash sandbox, a browser, an editor, an IPython kernel, all under one runtime. This is what you would actually *deploy*; MetaGPT is what you would actually *reason about*. OpenHands builds a MetaGPT-like team but inside a real runtime.

Reflexion (NeurIPS 2023)

Shinn et al. let an agent keep a verbal *self-reflection* after every failed trial. Next time, it reads its own past reflections and decides differently. No weight updates - just text in a memory buffer. Drops onto MetaGPT like a memory layer.

Voyager (NeurIPS 2023)

Wang et al. dropped GPT-4 into Minecraft with three modules: a curriculum generator that proposes the next hardest achievable task, a skill library that stores reusable code, and a reflection module. The agent grows its own toolkit forever. This is the skills-as-a-knowledge-graph idea, conceptually close to the Hermes skill system.

The meta-takeaway

Every one of these papers can be summarized as: *give the LLM a better loop*. MetaGPT gives it a team. SWE-agent gives it better tools. OpenHands gives it a runtime. Reflexion gives it memory. Voyager gives it lifelong growth. The frontier of 2026 harness-engineering is putting all of these together.

3. The MetaGPT Paper - Read-Along Guide

If you are going to teach this paper to others, here is how to read it in one sitting. The PDF is in the project folder: *02_metagpt.pdf*.

Section 1 Introduction - what problem they solve

Existing single-agent LLM systems fail at complex software engineering because they have no division of labour and no standard procedures. Humans solve this with SOPs (PM writes specs, Architect designs, Engineer codes, QA tests). MetaGPT claim: encoding SOPs as prompt sequences and running them across role-specialised agents gets you 3.6x better and 10x cheaper than GPT-4 single-agent on HumanEval pass@1.

Section 2 Related Work - where MetaGPT sits

- Single-agent (AutoGPT, BabyAGI): no structure, explodes in cost
- Multi-agent role-play (CAMEL): two agents, no SOP
- Society of Mind (Minsky, 1986): theoretical, no LLM implementation
- MetaGPT is the first to put SOPs into a multi-agent LLM system

Section 3 Framework - the architecture we re-implement

The MetaGPT architecture has three layers:

- **Environment** - a shared space where messages are published and subscribed. (Our MessagePool in `schema.py`.)
- **Agents (Roles and Profiles)** - a Role has a name, a profile (Senior PM), and a set of Actions it can execute. (Our Role class in `roles.py`.)
- **Actions** - the verbs. `WritePRD`, `WriteDesign`, `WriteCode`, `WriteTest`. Each takes context, emits a Message.

The *message* object is the key abstraction. It carries `role`, `content`, `cause_by`, `msg_id`, `created_at`. Every agent sees every message but only acts on the ones its SOP needs.

Section 3.4 Standard Operating Procedures - the meta-pattern

An SOP in MetaGPT is the directed-acyclic-graph of actions. The canonical SOP is: *WritePRD then WriteDesign then WriteCode then WriteTest*. Our `DEFAULT_SOP` list and `make_canonical_team` in `roles.py` implement exactly this.

Section 4 Experiments - what MetaGPT was measured on

- **HumanEval pass@1**: 85.9% with GPT-3.5 vs 67.8% single-agent (cost reduced 10x)
- **HumanEval-ET**: higher-quality pass when full SOP team runs
- **MBPP**: similar pattern; SOP team consistently wins
- **Software development benchmark**: full project (3.6x quality of single-agent)

Section 5 Discussion - limitations and future

MetaGPT does not learn across runs (that is where Reflexion and Voyager extend it). It assumes humans provide the SOP; it does not invent one. These limits define exactly the open problems the 2026 harness-engineering papers are trying to solve.

4. Our Reimplementation - File-by-File Walkthrough

Every line in MetaGPT-Mini maps to a concept in the paper. Here is what each file does, why it exists, and how to read it.

metagpt_mini/llm.py - LLM client (102 LOC)

The only file that knows about an LLM provider. Wraps the OpenAI Chat Completions API. Anything that exposes an OpenAI-compatible endpoint (MiniMax-M3, Qwen3-8B-Flash, Ollama, LM Studio) works.

```
class LLM:
    def __init__(self, model=None, base_url=None, api_key=None, ...):
        self.model = model or os.getenv('LLM_MODEL', 'MiniMax-M3')
        self.base_url = base_url or os.getenv('LLM_BASE_URL')
        self.api_key = api_key or os.getenv('LLM_API_KEY')
        ...
        self._client = OpenAI(api_key=..., base_url=...)

    def chat(self, messages, *, json_mode=False):
        # single-turn call, with cost accounting
        t0 = time.perf_counter()
        resp = self._client.chat.completions.create(...)
        self.usage.record(usage.prompt_tokens,
                        usage.completion_tokens, ...)
        return resp.choices[0].message.content

    def system_user(self, system, user, *, json_mode=False):
        return self.chat([ChatMessage('system', system),
                        ChatMessage('user', user)])
```

Key idea: self.usage accumulates tokens and an estimated cost. Team.run checks usage_cost() against MAX_BUDGET_USD between every role and stops if the budget is exceeded. This is a production hardening you will not find in toy agent demos.

metagpt_mini/schema.py - Message + MessagePool (54 LOC)

The substrate. Every piece of state in MetaGPT is a Message in a MessagePool. Two small classes, but the entire system runs on them.

```
@dataclass
class Message:
    role: str          # who emitted: ProductManager, Engineer, ...
    content: str        # the actual text (PRD, design doc, code, ...)
    cause_by: str       # which Action produced it
    msg_id: str         # short uuid
    created_at: str      # isoformat UTC

class MessagePool:
    def publish(self, msg): ...
    def history(self): ...
    def of_role(self, name): ... # messages from a role
    def by_action(self, cause_by): ... # messages from an action
```

Key idea: cause_by is the traceable SOP link. If you ever ask why did the Engineer write this code, you can read the chain: UserInput, WritePRD, WriteDesign, WriteCode.

metagpt_mini/actions.py - The four verbs (145 LOC)

Each Action is a dataclass with one method: run(pool) returns a Message. Read from the pool (e.g. the last PRD message), call the LLM with a structured system prompt, write the result back into the pool.

```

@dataclass
class WritePRD:
    llm: LLM
    cause_by: str = 'WritePRD'
    role_name: str = 'ProductManager'

    def run(self, requirement, pool):
        out = self.llm.system_user(PRD_SYSTEM, requirement)
        msg = Message(role='ProductManager',
                      content=out,
                      cause_by='WritePRD')
        pool.publish(msg)
        return msg

```

Four actions in canonical SOP order: WritePRD, WriteDesign, WriteCode, WriteTest. Each one is ~30 lines. Reading them top-to-bottom shows the entire workflow.

metagpt_mini/roles.py - The four agents (45 LOC)

```

@dataclass
class Role:
    name: str          # ProductManager
    profile: str        # Senior PM
    actions: list       # [WritePRD], [WriteDesign], etc.

    def run(self, pool):
        outs = []
        for action_factory in self.actions:
            action = action_factory() # instantiate action with shared llm
            msg = action.run(pool)
            outs.append(msg)
        return outs

def make_canonical_team(llm):
    return [
        Role('ProductManager', 'Senior PM', [lambda: WritePRD(llm)]),
        Role('Architect', 'Senior Architect', [lambda: WriteDesign(llm)]),
        Role('Engineer', 'Senior Backend Eng', [lambda: WriteCode(llm)]),
        Role('QA', 'Senior QA Engineer', [lambda: WriteTest(llm)]),
    ]

```

metagpt_mini/team.py - The orchestrator (84 LOC)

Owns the MessagePool, holds the Roles, runs them in SOP sequence, prints rich console panels, checks the budget between every role.

```

class Team:
    def run(self, requirement, budget_usd=None):
        seed = Message(role='User', content=requirement,
                      cause_by='UserInput')
        self.pool.publish(seed)

        for role in self.roles:
            if self.usage_cost() > budget:
                console.print('Budget blown - stopping.')
                break
            console.rule(role.name)
            for msg in role.run(self.pool):
                console.print(Panel(msg.content,
                                   title=f'{role.name} -> {msg.cause_by}'))

        return self.pool

```

examples/build_cli_app.py - The end-to-end demo (80 LOC)

The script you run. Loads `.env`, instantiates an LLM, builds a team, runs the SOP on a one-line requirement, then saves the four artifacts (`PRD.md`, `design.md`, `app.py`, `test_app.py`) to `output/`.

This is what you will demo live in class.

5. The SOP Trace - What You Will See Live

When you run the demo, here is the actual sequence of events your students will watch. Numbered for ease of narration.

Step 1 - User requirement enters as a Message

```
Message(role='User',
        content='Build a CLI todo app in Python with add, list,
        complete, delete, persist to JSON, priorities, due dates.',
        cause_by='UserInput')
```

Step 2 - ProductManager runs WritePRD

The LLM is given a system prompt that says you are a senior PM. It produces a structured PRD with Goal, User Stories, Functional + Non-Functional Requirements, Out-of-scope. The message enters the pool with `cause_by=WritePRD`.

Step 3 - Architect runs WriteDesign

Reads the PRD from the pool (`_last_of(pool, WritePRD)`). Produces a design with modules, data model, function signatures. Enters the pool with `cause_by=WriteDesign`.

Step 4 - Engineer runs WriteCode

Reads the design. Produces a complete, runnable Python module with type hints and a `__main__` demo. Enters the pool with `cause_by=WriteCode`.

Step 5 - QA runs WriteTest

Reads both design and code. Produces a pytest test module covering happy path, edge cases, and a failure case. Enters the pool with `cause_by=WriteTest`.

Step 6 - Output is saved

The demo script extracts each message and writes them to disk as:

- `output/01_prd.md` - the Product Requirements Document
- `output/02_design.md` - the technical design
- `output/03_app.py` - the runnable app (code fences stripped)
- `output/04_test_app.py` - the pytest module

Step 7 - Run it

```
cd /Users/jkm/Projects/metagpt-mini
source .venv/bin/activate
python examples/build_cli_app.py

# Then test the result:
cd output
python 03_app.py add 'Buy milk' --priority high --due 2026-09-07
python 03_app.py list
pytest 04_test_app.py -v
```

If the SOP is correct, you have a working CLI todo app built end-to-end by 4 LLM agents in 4 LLM calls. Total wall time: 30-90 seconds with MiniMax-M3. Total cost: roughly \$0.05 - \$0.20.

6. Windows + MiniMax-M3 Setup - Step by Step

You will demo this on your Windows machine. The setup is the same as on Mac, with two Windows-specific tweaks (Git Bash shell, .env file). Total time: 15 minutes if Python and Git are already installed.

6.1 Prerequisites (Windows)

- **Python 3.11+** - install from python.org, *check Add to PATH* at install time
- **Git** - install from git-scm.com, pick Git Bash as default terminal
- **uv** - the Python package manager. In Git Bash, run: `pip install uv`
- **MiniMax-M3 API key** - your existing key. Keep it secret.

6.2 Clone the repo

```
# In Git Bash (or PowerShell - both work)
mkdir ~/Projects
cd ~/Projects
git clone https://github.com/mannuking/metagpt-mini.git
cd metagpt-mini
```

6.3 Set up the Python environment

```
# In Git Bash on Windows
cd ~/Projects/metagpt-mini
uv venv                                # creates .venv/
source .venv/Scripts/activate          # Windows path: Scripts/, not bin/
uv pip install -e .                    # installs package + dependencies
```

6.4 Configure your .env (the critical step)

```
# Copy the template
cp .env.example .env

# Edit it (use notepad, vscode, or vim)
notepad .env

# Fill in these three lines:
# LLM_API_KEY=sk-your-minimax-key-here
# LLM_BASE_URL=https://api.minimax.chat/v1
# LLM_MODEL=MiniMax-M3

# Save and close. NEVER commit the .env file
# (it is already in .gitignore).
```

6.5 Verify the setup

```
# Run the schema tests - these do not need the LLM
pytest tests/ -v

# Expected output:
# tests/test_schema.py::test_message_roundtrip PASSED
# tests/test_schema.py::test_pool_publish_and_filter PASSED
# tests/test_schema.py::test_pool_ordering PASSED
# ===== 3 passed in 0.01s =====
```

6.6 Run the demo

```
# Source the venv (every new shell needs this)
source .venv/Scripts/activate

# Run the canonical demo
```

```
python examples/build_cli_app.py
```

```
# After the run, test the output:
cd output
python 03_app.py --help
pytest 04_test_app.py -v
```

6.7 Common Windows gotchas

- **Path separators:** Windows uses Scripts/ where Mac/Linux use bin/. Source the venv with `source .env/Scripts/activate` in Git Bash.
- **Line endings:** Git on Windows may convert LF to CRLF. If you see syntax errors in agent output, run `git config core.autocrlf false` then re-clone.
- **.env not loaded:** python-dotenv reads .env from the current working directory. Always cd into the project folder before launching Python.
- **API key leakage:** if you accidentally commit .env, rotate the key immediately (GitHub will also auto-redact it in push protection, but treat it as exposed).
- **Slow first run:** the first LLM call may take 5-10 seconds for TLS handshake + cold-start; subsequent calls in the same run are 1-3s.
- **Rich console on older Windows terminals:** if the colored output looks broken in cmd.exe, run inside Windows Terminal or Git Bash.

6.8 Switching models without code changes

Because the framework only talks to an OpenAI-compatible API, switching to Qwen3-8B-Flash, GPT-4, or local Ollama is just a .env change:

```
# Qwen3-8B-Flash (Chinese cloud, OpenAI-compat mode)
LLM_API_KEY=sk-qwen-key
LLM_BASE_URL=https://dashscope.aliyuncs.com/compatible-mode/v1
LLM_MODEL=qwen3-8b-flash

# Local Ollama (open-source, runs on your GPU)
LLM_API_KEY=ollama # any non-empty string
LLM_BASE_URL=http://localhost:11434/v1
LLM_MODEL=qwen2.5-coder:7b # or any tag you have pulled

# LM Studio (local desktop app)
LLM_API_KEY=lm-studio
LLM_BASE_URL=http://localhost:1234/v1
LLM_MODEL=qwen2.5-7b-instruct
```

Appendix A - Glossary for the Class

Agent - An LLM-driven entity that reads context, decides, and acts. In MetaGPT, a Role is an Agent.

Action - The unit of behaviour. An Action takes context and emits a Message.

Role - A named bundle of Actions + a profile. Roles are reusable across runs.

SOP - Standard Operating Procedure. The directed graph of Actions that defines a workflow.

Message - An event emitted by an Action. Carries role, content, cause_by, msg_id, timestamp.

Message Pool - The shared bus through which agents communicate.

ACI - Agent-Computer Interface (SWE-agent term). Purpose-built verbs instead of raw shell.

Harness - The runtime + tooling + environment around an LLM. MetaGPT is a harness; OpenHands is a harness; Hermes Agent is a harness.

Self-Reflection - An agent own critical assessment of its output (Reflexion idea).

Skill Library - A growing collection of reusable code/tool definitions an agent writes and reuses (Voyager idea).

Appendix B - Citations (BibTeX)

```
@inproceedings{hong2024metagpt,
  title = {MetaGPT: Meta Programming for A Multi-Agent Collaborative Framework},
  author = {Hong, Sirui and Chen, Jonathan and Zhu, Ceyao and others},
  booktitle = {ICLR},
  year = {2024},
  note = {Oral presentation, top 1.2 percent, ranked no.1 in LLM-Agent category}
}

@inproceedings{yang2024sweagent,
  title = {SWE-agent: Agent-Computer Interfaces Enable Automated Software Engineering},
  author = {Yang, John and others},
  booktitle = {NeurIPS},
  year = {2024}
}

@inproceedings{wang2024openhands,
  title = {OpenHands (formerly OpenDevin): An Open Platform for AI Software Developers},
  author = {Wang, Xingyao and others},
  booktitle = {COLM},
  year = {2024},
  note = {Oral presentation}
}

@inproceedings{shinn2023reflexion,
  title = {Reflexion: Language Agents with Verbal Reinforcement Learning},
  author = {Shinn, Noah and Cassano, Federico and Gopinath, Ashish and others},
  booktitle = {NeurIPS},
  year = {2023}
}

@inproceedings{wang2023voyager,
  title = {Voyager: An Open-Ended Embodied Agent with Large Language Models},
  author = {Wang, Guanzhi and Xie, Yuqi and Jiang, Yunfan and others},
  booktitle = {NeurIPS},
  year = {2023}
```

}

Appendix C - File-by-file cheat sheet

File	LOC	What it does
metagpt_mini/__init__.py	1	Package marker, version.
metagpt_mini/llm.py	102	LLM client + cost tracker. Only file that talks to an API.
metagpt_mini/schema.py	54	Message + MessagePool. The substrate.
metagpt_mini/actions.py	145	WritePRD, WriteDesign, WriteCode, WriteTest.
metagpt_mini/roles.py	45	Role + make_canonical_team.
metagpt_mini/team.py	84	Team orchestrator with budget guard + rich UI.
metagpt_mini/utils.py	1	Cost-tracker re-export.
examples/build_cli_app.py	80	End-to-end runnable demo. The script you teach from.
tests/test_schema.py	34	3 unit tests. Run before any LLM call.
pyproject.toml	-	Project metadata. Edit name/desc here.
README.md	-	GitHub README. Top-level orientation.
.env.example	-	Copy to .env, fill in LLM_API_KEY.
docs/build_pdf.py	-	Generates this PDF.
docs/MetaGPT-Mini-Learning-Guide.pdf	-	This document.